

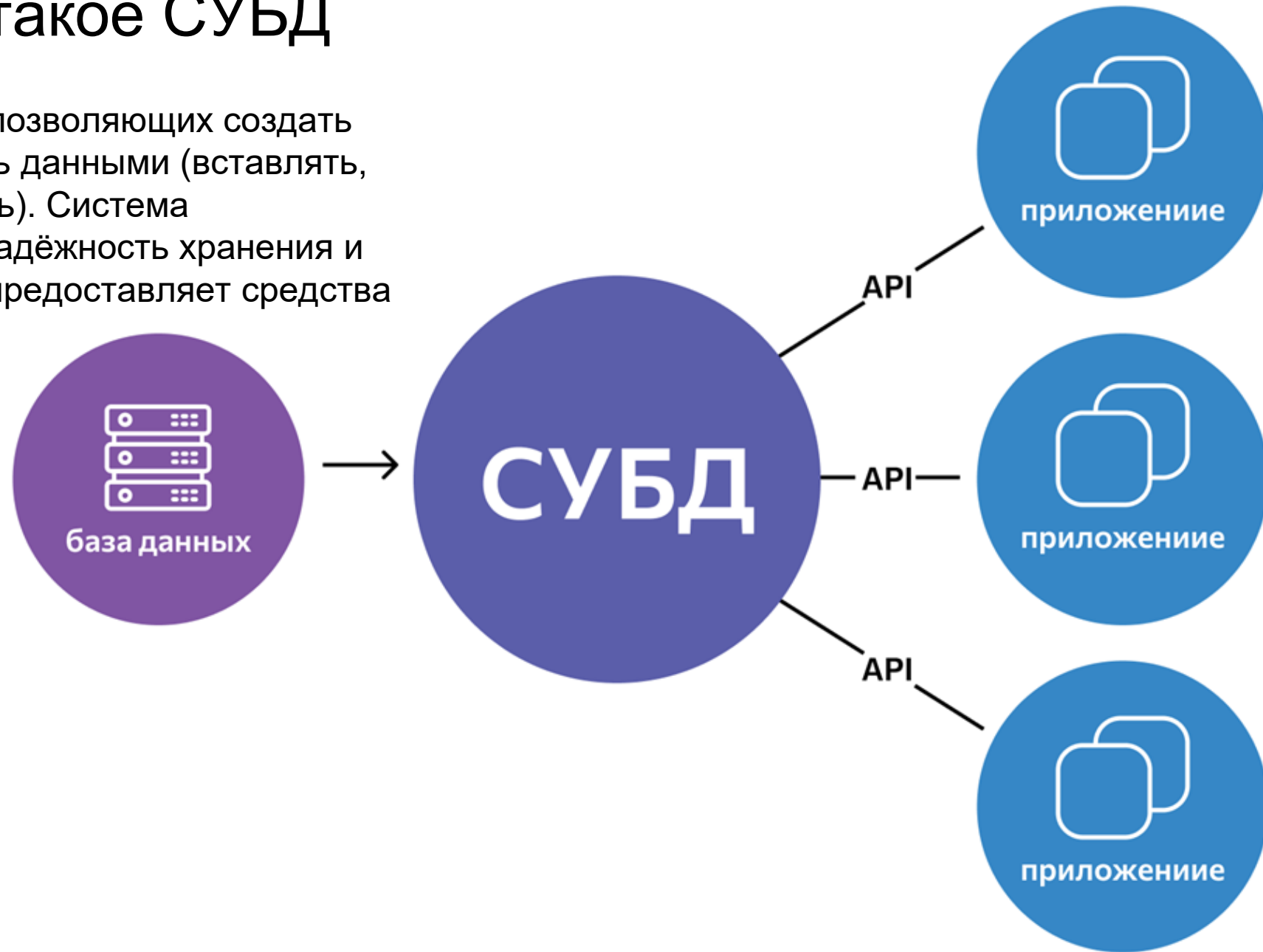
# Базовые информационные процессы и технологии

## Тема №3 MongoDB

Докладчик:  
Нетбай Г.В., ведущий инженер кафедры ВММБ

# Что такое СУБД

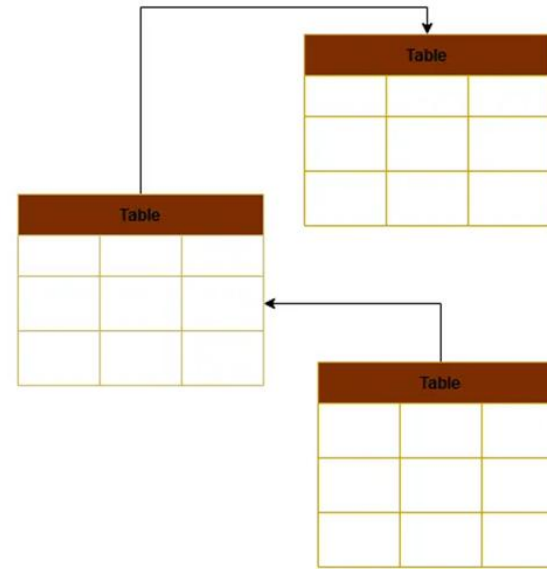
**СУБД** — комплекс программ, позволяющих создать базу данных и манипулировать данными (вставлять, обновлять, удалять и выбирать). Система обеспечивает безопасность, надёжность хранения и целостность данных, а также предоставляет средства для администрирования БД.



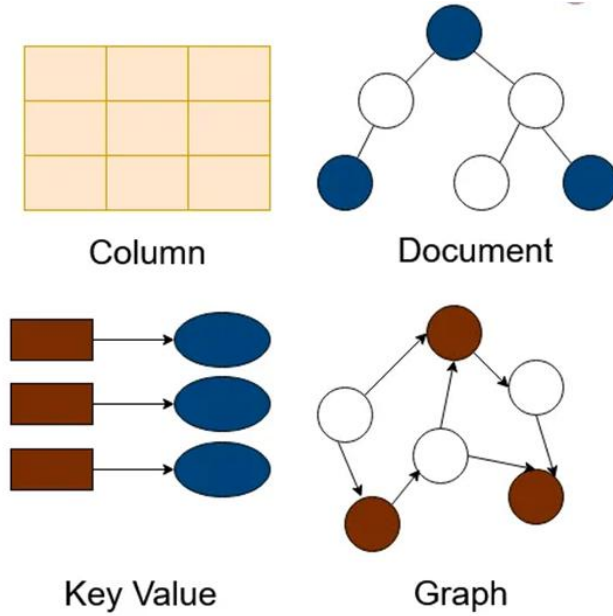
# Две категории баз данных

**SQL (Structured Query Language)** — это язык структурированных запросов, используемый для управления и манипулирования реляционными базами данных. SQL-базы данных применяются там, где необходимо хранить и управлять данными структурированной природы, например, информацией о продуктах, покупателях и оформленных заказах в магазине.

**NoSQL (Not Only SQL)** — это широкий термин, который относится к нереляционным моделям баз данных. NoSQL-базы данных применяются, когда необходимо хранить данные неструктурированной природы, например, большие объёмы текстовых данных, изображения и видео.



SQL Databases



No SQL Databases

# Виды БД по структуре организации данных



PostgreSQL



ClickHouse



Реляционные

Документо-  
ориентированные

Ключ-значение

Графовые

Колоночные

Векторные



mongoDB



redis



Milvus

Данные хранятся в виде таблиц, которые связаны друг с другом при помощи ключей. Такая архитектура использует строчное хранение данных и нужна для создания строгой структуры.

## Транзакционность

- группа операций применяется к данным целиком или не применяется совсем

Данные необходимо обновлять часто, а структуру – нет

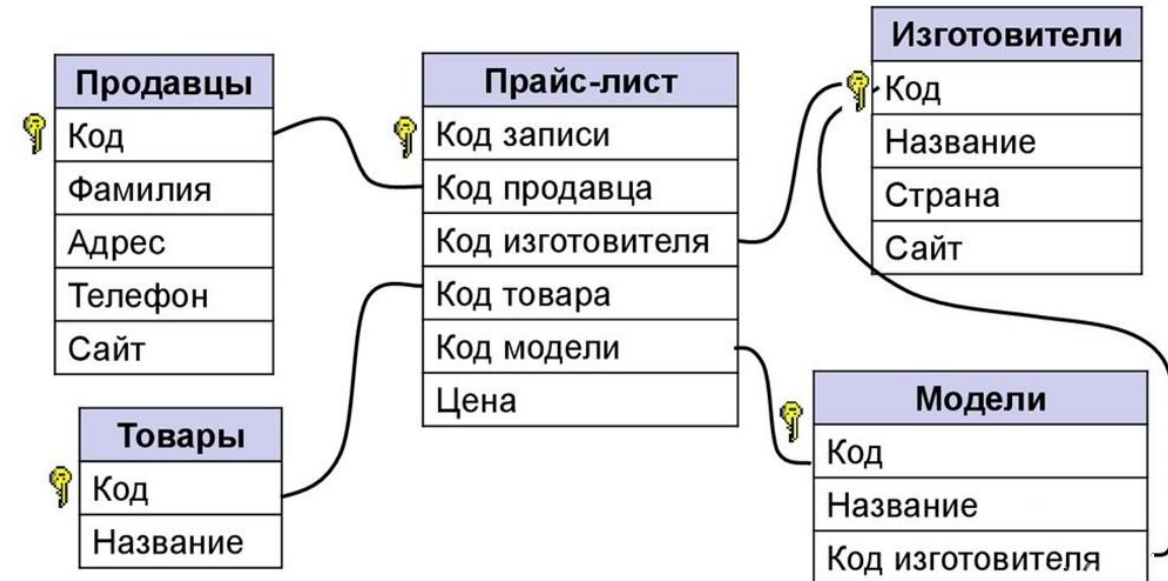
Возможно применять индексирование к данным

Непродолжительные по времени операции над данными

- нет длительных запросов аналитики (от минут до часов)

Объём данных не превышает нескольких терабайт

- есть исключения – Timescale DB



Каждый элемент базы данных использует состоит из значения и уникального ключа. Этот способ хранения похож на тип данных «словарь» в языках программирования.

Базы данных типа «ключ — значение» имеют вид хеш-таблиц, в которых у каждой записи — только один индекс. При этом жёстких ограничений по структуре элементов нет.

Области применения баз данных типа «ключ — значение» — очереди, кэш и хранение актуального состояния конфигурации динамической инфраструктуры.

| Key       | Value                                                                                                                                               |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| "India"   | {"B-25, Sector-58, Noida, India – 201301"}                                                                                                          |
| "Romania" | {"IMPS Moara Business Center, Buftea No. 1, Cluj-Napoca, 400606", City Business Center, Coriolan Brediceanu No. 10, Building B, Timisoara, 300011"} |
| "US"      | {"3975 Fair Ridge Drive. Suite 200 South, Fairfax, VA 22033"}                                                                                       |



Хранят информацию в виде «документов», а не таблиц и строк.

Главное преимущество — возможность хранить информацию без строгого ограничения по структуре.

В отличие от хранилищ «ключ-значение», документоориентированные БД включают метаданные, связанные с хранимым содержимым, что даёт возможность делать запросы на его основе.

Например, в указанном примере можно найти все документы, в которых поле «City» равно «Noida».

```
{officeName: "3Pillar Noida",  
  {Street: "B-25", City: "Noida",  
    State: "UP", Pincode: "201301"}  
}  
{officeName: "3Pillar Timisoara",  
  {Boulevard: "Coriolan Brediceanu No. 10",  
    Block: "B, Ist Floor",  
    City: "Timisoara", Pincode: "300011"}  
}  
{officeName: "3Pillar Cluj",  
  {Latitude: "40.748328",  
    Longitude: "-73.985560"}  
}
```



**Couchbase**



**mongoDB**

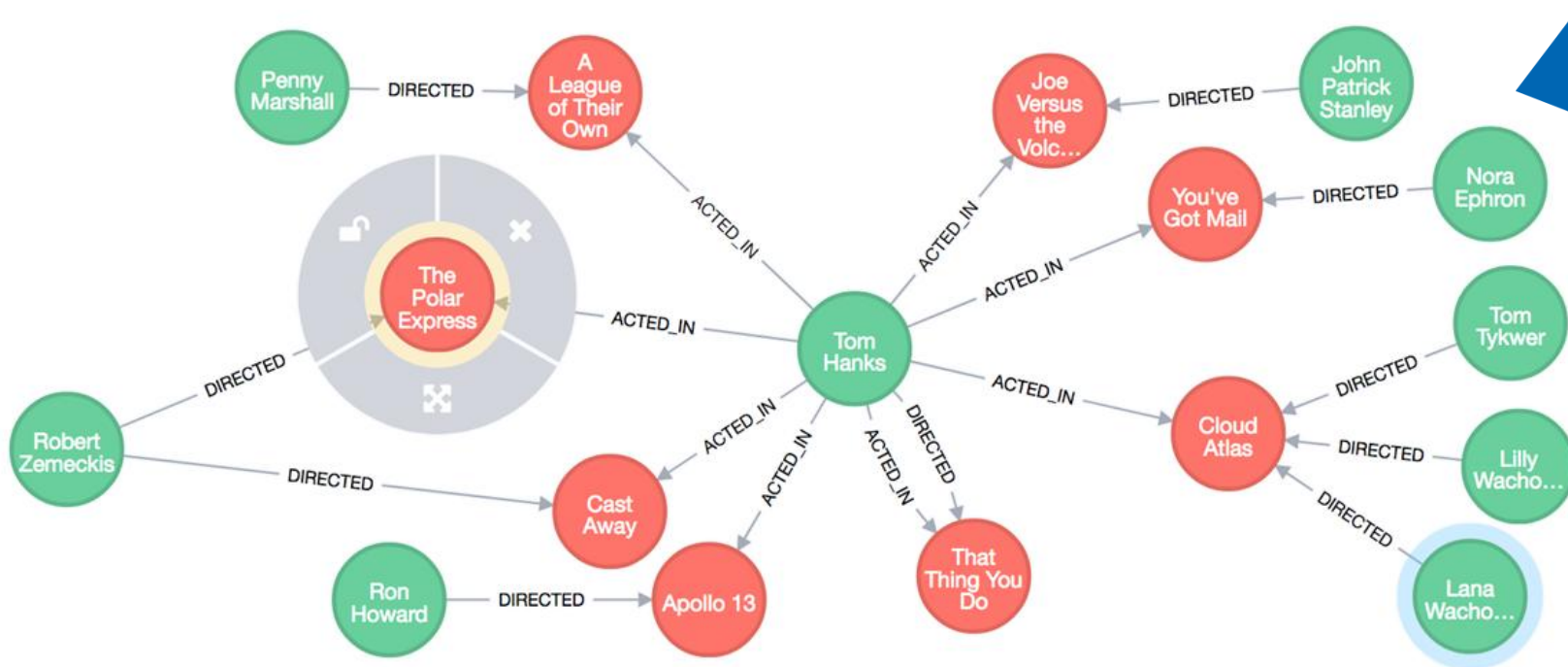


**Firebase**



**Графовые.** Каждый узел графовой БД содержит в себе данные. Для хранения взаимосвязи между узлами используют рёбра. В них хранятся начальный и конечный узлы, направление и тип связи. Например, «родитель — потомок».

Применяются для моделирования социальных графов (например, в системах обнаружения мошенничества), для хранения семантических сетей и т.п. В задачах с естественной графовой структурой данных могут существенно превосходить по производительности реляционные, а также имеют преимущества в наглядности представления и простоте внесения изменений в схему данных.



**Amazon  
Neptune**



**Колоночные.** В них данные хранятся последовательно в виде разрозненных колонок. В каждой колонке хранится по свойству сущности. Колонки могут быть проанализированы независимо. Это повышает скорость обработки данных. Колоночные БД позволяют загружать новые данные большим потоком. При этом в некоторых реализациях колоночных БД изменять или удалять данные нельзя.

Аналитики используют колоночные БД для хранения событий и выполнения их анализа. Например, чтобы вычислять на основе показанной рекламы и других сведениях о пользователе, что ему может понадобиться.

| ID<br>пользователя | Время<br>пребывания | ID<br>рекламы |
|--------------------|---------------------|---------------|
| xxxxxx             | 120                 | 2             |
| yyyyy              | 90                  | 12            |
| zzzzz              | 500                 | 7             |



Google  
Big Query



ClickHouse



druid

Данные хранятся в виде многомерных векторов, измерения которых представляет характеристики элементов выборки.

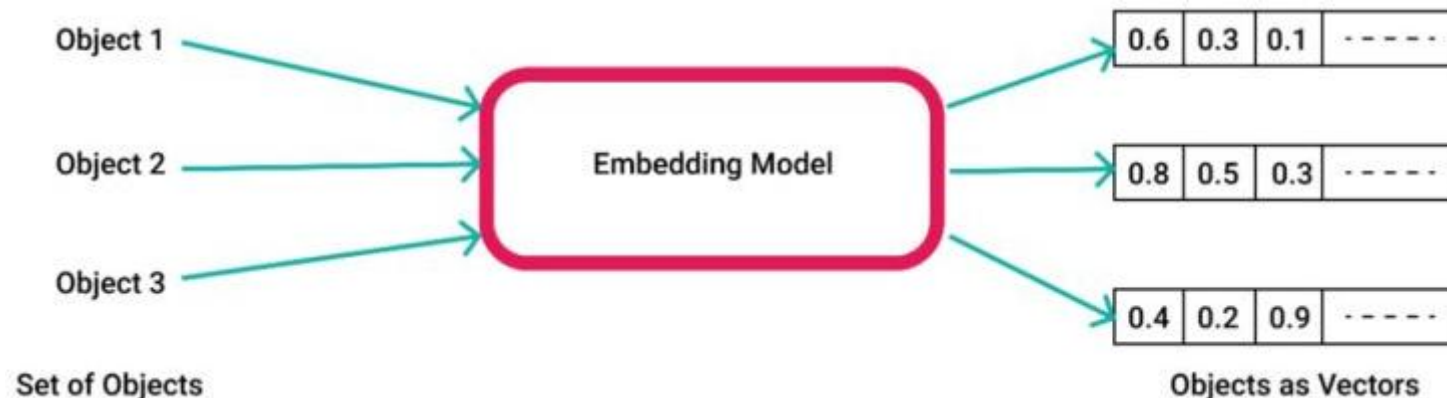
Векторы могут иметь различное количество измерений, в зависимости от сложности данных.

Для преобразования данных, включая текст, изображения, аудио и видео, в эти векторы используются различные методы, например, модели машинного обучения.

Векторные БД проектируются с учётом необходимости эффективно извлекать данные на основе близости или сходства векторов. Это позволяет выполнять поиски на основе семантической и контекстуальной значимости.



**Weaviate**



# Виды БД по структуре организации данных

| Параметры          | Реляционные                                                                      | Ключ-значение                              | Документо-ориентированные                | Графовые                                         | Колоночные                                                                                                                                                             | Векторные                                  |
|--------------------|----------------------------------------------------------------------------------|--------------------------------------------|------------------------------------------|--------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| Особенности        | + Транзакции<br>+ Связи между сущностями<br>Подходят для фиксированной структуры | Эффективная работа с данными ключ-значение | Работа с данными с переменной структурой | Эффективная работа с графовыми структурами       | Обновление, удаление, связи между записями, транзакции – не все СУБД поддерживают<br>Эффективная обработка большого объема данных: вставка потоком или пачками, анализ | Быстрые операции поиска по векторам        |
| Область применения | Описание сущностей и их взаимосвязей                                             | Кэширование<br>Хранение конфигурации       | Хранение данных с переменной структурой  | Семантические сети<br>Описание социальных связей | Аналитика большого объема данных                                                                                                                                       | Хранение embedding                         |
| Примеры реализации | PostgreSQL<br>MySQL<br>Oracle<br>SQLite                                          | Redis<br>etcd<br>DynamoDB<br>Memcached     | MongoDB<br>Couchbase<br>Firebase         | Infinitegraph<br>Neo4j<br>Amazon Neptune         | ClickHouse<br>Apache Druid<br>Google Big Query<br>Teradata                                                                                                             | Chrome<br>MilvusDB<br>Pinecone<br>Weaviate |

# MongoDB. Основные особенности.

**MongoDB** — документоориентированная система управления базами данных, не требующая описания схемы таблиц. Считается одним из классических примеров NoSQL-систем, использует JSON-подобные документы и схему базы данных. Написана на языке C++.

## Ad-hoc запросы

- Возвращают только запрошенные поля

## Поиск с использованием регулярных выражений

## Запросы случайного подмножества данных

## Индексы

- Возможность ускорить выборку данных при помощи индексации

## Валидаторы

- Есть возможность ограничить структуру документа при помощи JSON схемы

## JavaScript фильтрация данных

- Использование при поиске условий фильтрации на JS

## Кластеризация и шардирование

- Поддержка master-slave для отказоустойчивости
- Горизонтальное масштабирование



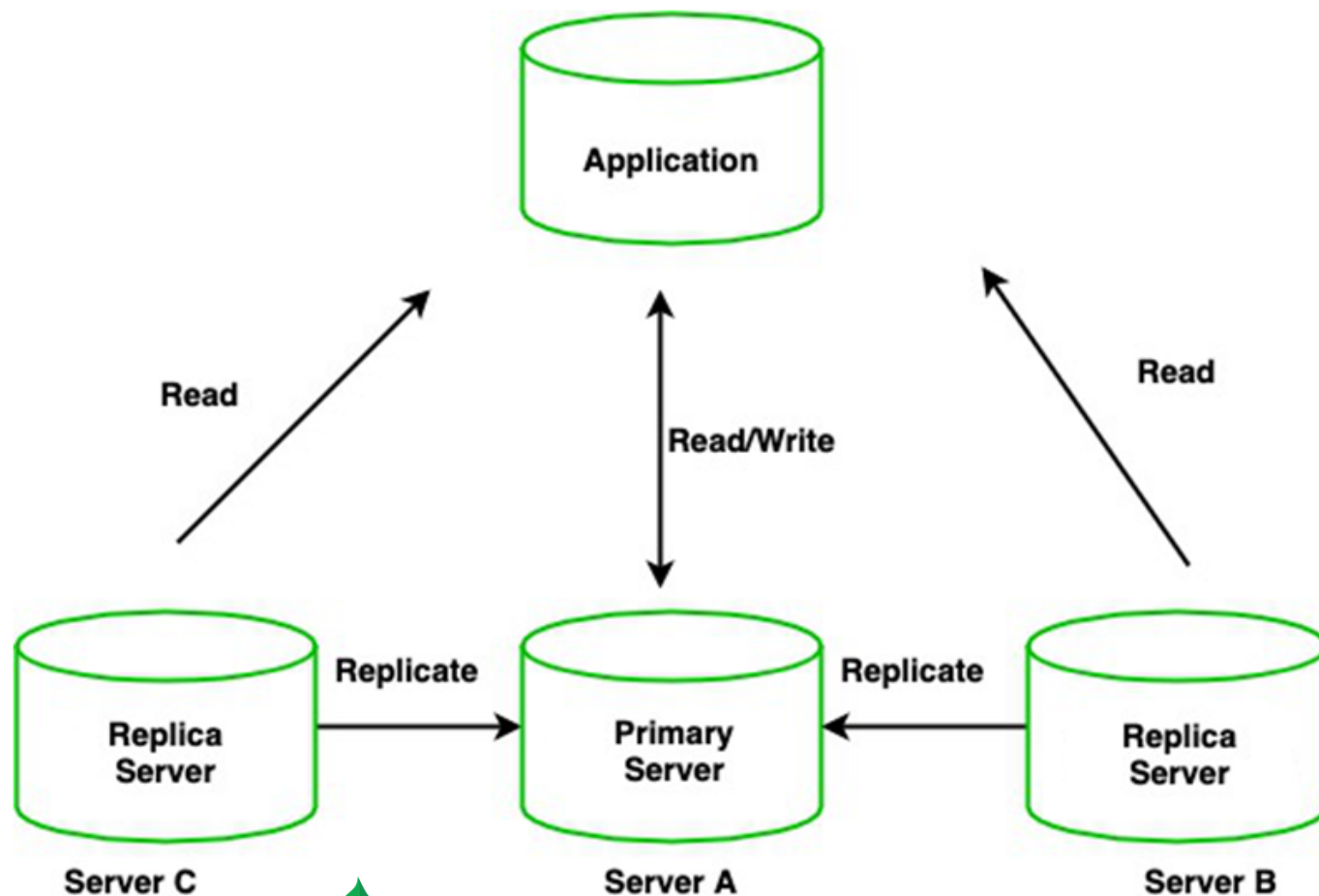
# mongoDB

# MongoDB. Обеспечение отказоустойчивости.

Для обеспечения отказоустойчивости, система поддерживает кластеризацию. Каждая реплика в кластере может в любой момент выступить в роли основной или вспомогательной.

В случае, когда основная реплика дает сбой, набор реплик проводит выбор, которая из реплик должна стать основной.

Все операции записи производятся с основной репликой. По-умолчанию, чтение тоже производится с основной реплики. Второстепенные реплики могут применяться для балансировки операций чтения.



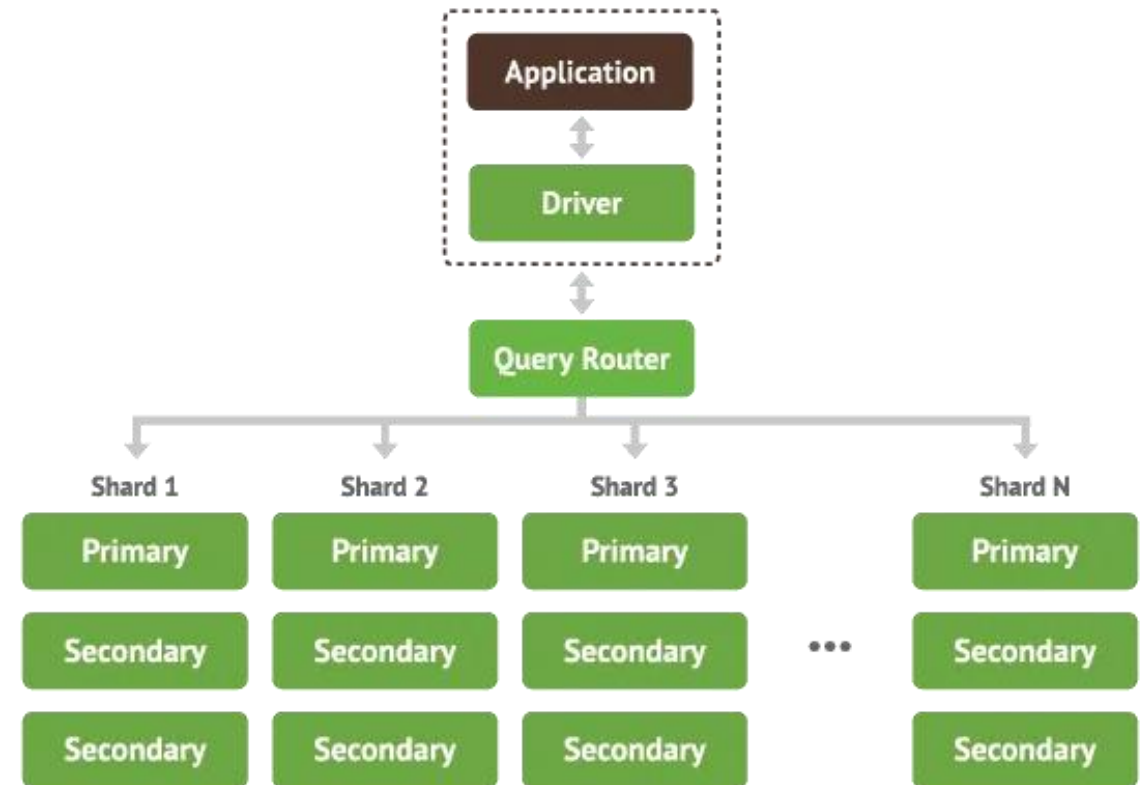
# mongoDB

# MongoDB. Горизонтальное масштабирование.

Система масштабируется горизонтально, используя технику сегментирования объектов баз данных — распределение их частей по различным узлам кластера.

Администратор выбирает ключ сегментирования, который определяет, по какому критерию данные будут разнесены по узлам (в зависимости от значений хэша ключа сегментирования).

Благодаря тому, что каждый узел кластера может принимать запросы на обработку своей части данных, повышается производительность.



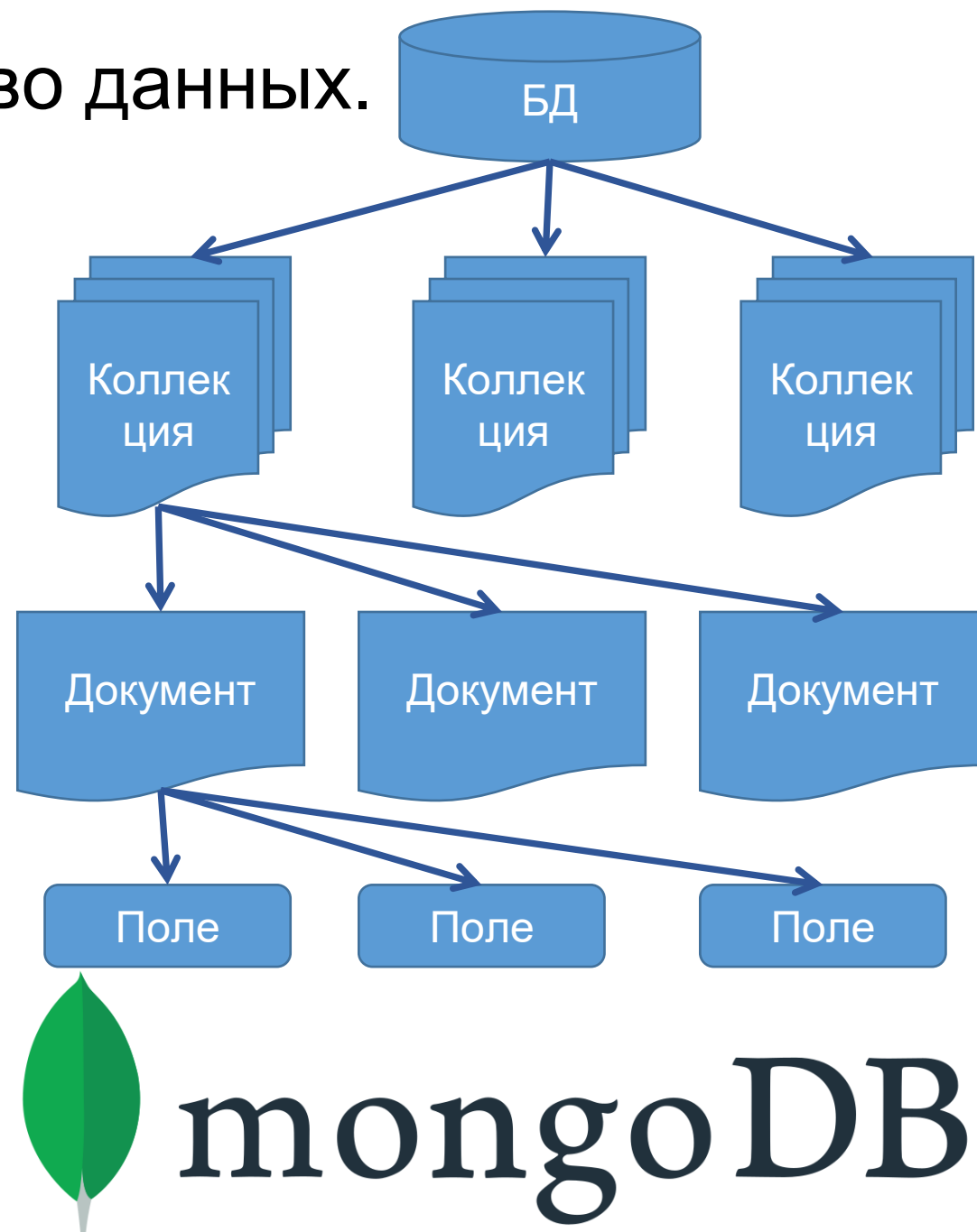
mongoDB

# MongoDB. Устройство данных.

В реляционных базах данных информация хранится в виде взаимосвязанных таблиц. Их структура жёстко задана, и менять её накладно. Строки каждой таблицы имеют одинаковый набор полей, данные обрабатывают с помощью запросов на языке SQL.

В MongoDB базы состоят из коллекций и документов — иерархических структур, содержащих пары «ключ — значение» (поля). Если проводить аналогии с реляционной базой, коллекции при таком способе хранения соответствуют таблицам, а документы — строкам. У документов нет строгой структуры. Они могут содержать разные наборы полей, причём различающиеся как по типу, так и по количеству.

Информация отформатирована в BSON — двоичной кодировке JSON-подобных документов. Это позволяет поддерживать данные типа Date и двоичных файлов, что невозможно в JSON.





# MongoDB. Основные типы данных.

## Нулевой тип

- Используется для обозначения нулевого значения и несуществующего поля
- `{"x": null}`

## Логический тип

- Используется для значений true и false
- `{"x": true}`

## Число

- По умолчанию используется 64-битные числа с плавающей точкой
- `{"x": 3.14}`
- `{"x": 3}`

## Целое число

- Для целых чисел существуют два типа данных: 4 и 8 байт
- `NumberInt` - 4 байта
- `NumberLong` - 8 байт
- `{"x": NumberInt("3")}`
- `{"x": NumberLong("3")}`

## Строка

- Любая строка символов в кодировке UTF-8.
- `{"x": "foobar"}`

## Дата

- MongoDB хранит даты в виде 64-битных целых чисел, обозначающих миллисекунды с момента эпохи Unix (1 января 1970 г.). Часовой пояс не сохраняется.
- `{"x": new Date()}`

## Массив

- Списки значений могут быть представлены в виде массивов.
- `{"x": ["a", "b"]}`

## Идентификатор объекта

- 12-байтовый уникальный идентификатор документа
- `{"x": ObjectId()}`

## Встраиваемый документ

- Документы могут содержать другие документы, встроенные в качестве значений в родительский документ.
- `{"x": {"foo": "a"}}`

## Двоичные данные

- Набор байтов.
- Двоичные данные – единственный способ сохранять строки не в формате UTF-8 в БД.



MongoDB может быть развёрнут в следующих конфигурациях:

- В кластере Kubernetes
- **При помощи Docker/Docker compose (предпочтительный вариант для выполнения задания)**
- Как самостоятельное приложение для Linux/Windows/MacOS

```
docker-compose.yml
1 services:
2   mongodb:
3     image: mongodb/mongodb-community-server:4.4-ubuntu2004
4     environment:
5       - MONGO_INITDB_ROOT_USERNAME=root
6       - MONGO_INITDB_ROOT_PASSWORD=root
7     ports:
8       - 27017:27017
9     volumes:
10      - ./data/db:/data/db
11      - ./data/configdb:/data/configdb
```

Указываем старую версию MongoDB, чтобы запустилось даже на процессоре без AVX

Логин и пароль суперпользователя

Тома, в которых будут храниться конфигурация и БД



# mongoDB

# MongoDB. Подключение к серверу.

Для подключения к серверу из Python можно использовать модуль PyMongo

Команда для установки:

```
pip install pymongo
```

Примеры работы с mongo из Python можно посмотреть по ссылке:

[https://www.w3schools.com/python/python\\_mongodb\\_getstarted.asp](https://www.w3schools.com/python/python_mongodb_getstarted.asp)

```
test_connection.py x
1  from pymongo import MongoClient
2
3  if __name__ == '__main__':
4      client = MongoClient(host="mongodb://root:root@localhost", port=27017)
5      client.test.test.insert_one({})
6
```

Имя хоста,  
на котором  
развёрнута  
СУБД

Порт сервера из  
docker compose  
для подключения

Выполняем тестовую  
операцию вставки,  
чтобы проверить, что  
всё работает

Логин и пароль  
пользователя



# mongoDB



# MongoDB. Создание БД.

```
create_db.py x
1 from pymongo import MongoClient
2
3 if __name__ == '__main__':
4     client = MongoClient(host="mongodb://root:root@localhost", port=27017)
5     db = client["database_1"]
6     db.test.insert_one({})
7     print(client.list_database_names())
```

Вставляем в БД в  
коллекцию test документ  
**БД создастся при  
первом обращении**

Выводим на экран  
список БД, чтобы  
убедиться, что БД  
создалась

Создаём в Python  
ссылку на БД, с  
которой будем  
работать

```
['admin', 'config', 'database_1', 'local', 'mydatabase', 'test']
```

Созданная БД



# mongoDB

# MongoDB. Создание коллекции.

```
create_collection.py x
1 from pymongo import MongoClient
2
3 if __name__ == '__main__':
4     client = MongoClient(host="mongodb://root:root@localhost", port=27017)
5     db = client["database_2"]
6     collection_1 = db["collection_1"]
7     collection_1.insert_one({})
8     print(db.list_collection_names())
```

Вставляем в БД в коллекцию  
collection\_1 документ  
**Коллекция создастся при  
первом обращении**

Выводим на экран список  
коллекций, чтобы убедиться, что  
коллекция создалась

Создаём в Python  
ссылку на коллекцию,  
с которой будем  
работать

```
live-coding\create_collection.py"
['collection_1']
Process finished with exit code 0
```

Созданная  
коллекция



# mongoDB

# MongoDB. Вставка элементов.

```
insert_one.py x
1 from pymongo import MongoClient
2
3 if __name__ == '__main__':
4     client = MongoClient(host="mongodb://root:root@localhost", port=27017)
5     db = client["database_2"]
6     collection_2 = db["collection_2"]
7     result = collection_2.insert_one({ "name": "John", "address": "Highway 37" })
8     print(result.inserted_id)
```

Получаем  
результат  
операции

Выводим на экран ID  
вставленного  
объекта

Вставляем в БД в коллекцию  
collection\_2 документ  
**Коллекция создастся при  
первом обращении**



```
live-coding\insert_one.py"
67ce4845d5c2bd143c1626e2

Process finished with exit code 0
```



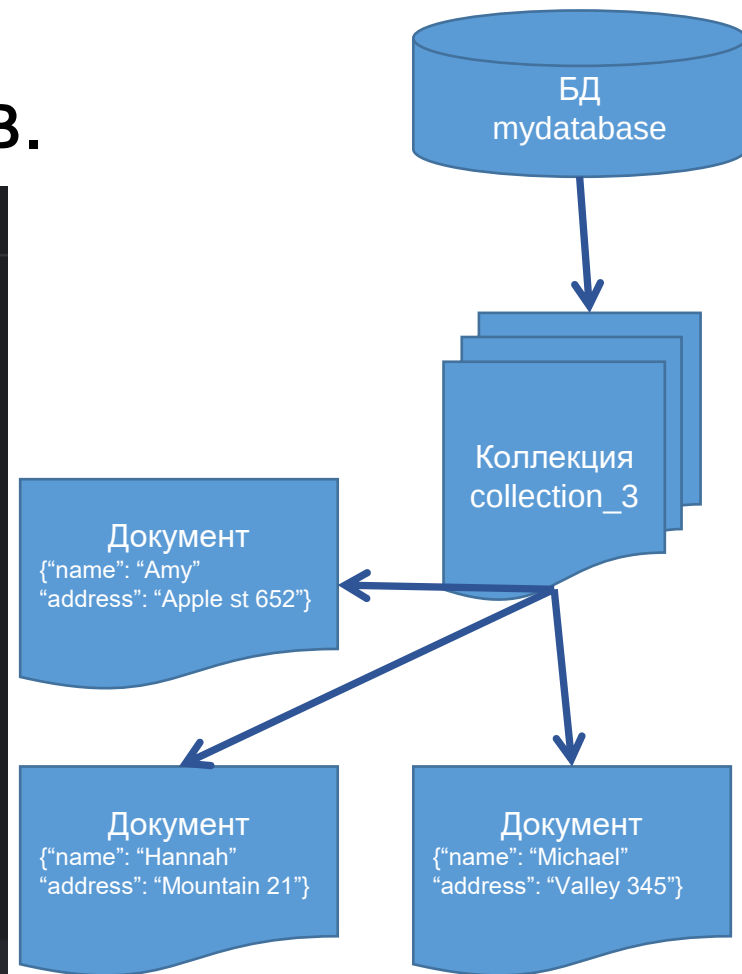
# mongoDB



```
insert_many.py x
1 from pymongo import MongoClient
2
3 if __name__ == '__main__':
4     client = MongoClient(host="mongodb://root:root@localhost", port=27017)
5     db = client["mydatabase"]
6     collection_3 = db["collection_3"]
7     result = collection_3.insert_many([
8         {"name": "Amy", "address": "Apple st 652"},
9         {"name": "Hannah", "address": "Mountain 21"},
10        {"name": "Michael", "address": "Valley 345"}
11    ])
12
13 print(result.inserted_ids)
```

Получаем результат  
операции

Выводим на экран ID  
вставленных  
объектов



Вставляем в БД в коллекцию collection\_3  
несколько документов сразу  
**Коллекция создается при первом  
обращении**

```
live-coding\insert_many.py"
```

```
[ObjectId('67ce498571c1f22d553d3fdf'), ObjectId('67ce498571c1f22d553d3fe0'), ObjectId('67ce498571c1f22d553d3fe1')]
```





# MongoDB. Вставка элементов.

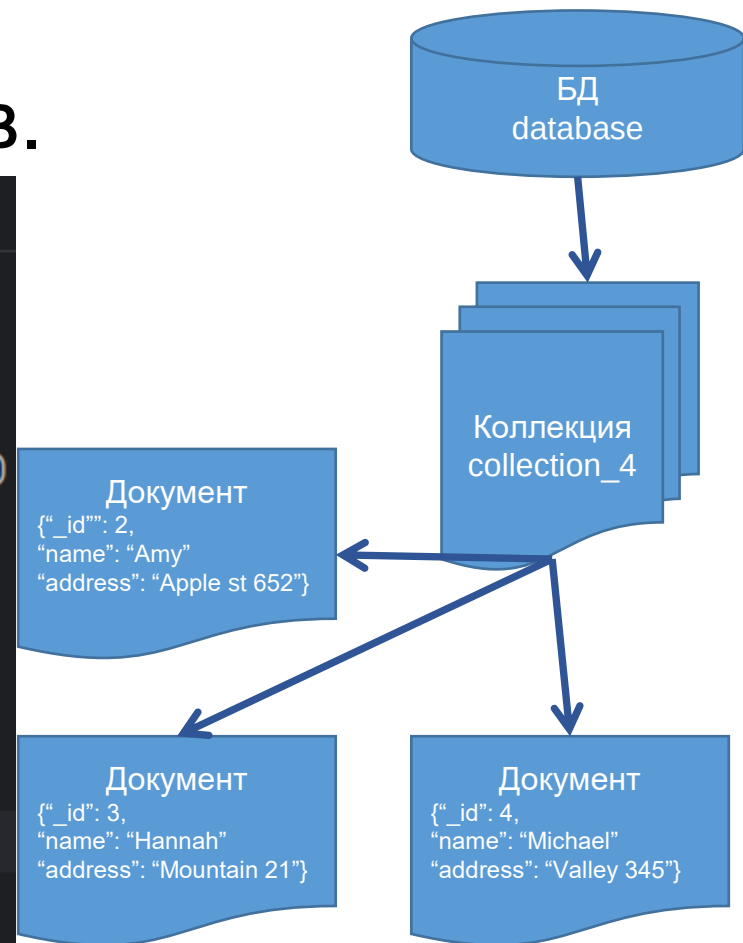
```
insert_many_2.py x
1 from pymongo import MongoClient
2
3 if __name__ == '__main__':
4     client = MongoClient(host="mongodb://root:root@localhost", port=27017)
5     db = client["database"]
6     collection = db["collection_4"]
7     result = collection.insert_many([
8         {"_id": 2, "name": "Amy", "address": "Apple st 652"},
9         {"_id": 3, "name": "Hannah", "address": "Mountain 21"},
10        {"_id": 4, "name": "Michael", "address": "Valley 345"}
11    ])
12
13 print(result.inserted_ids)
```

Получаем  
результат  
операции

Выводим на экран ID  
вставленных объектов

```
live-coding\insert_many_2.py"
[2, 3, 4]

Process finished with exit code 0
```



Вставляем в БД в коллекцию  
collection\_4 несколько документов сразу,  
задавая идентификаторы  
**Коллекция создается при первом  
обращении**



# mongoDB

# MongoDB. Выборочное получение полей.

```
get_part_columns.py x
rgij\Desktop\Тема №3 MongoDB\Theme 3 live-coding goClient
2
3 if __name__ == '__main__':
4     client = MongoClient(host="mongodb://root:root@localhost", port=27017)
5     db = client["database"]
6     collection = db["collection_7"]
7     collection.insert_many([
8         {"name": "Amy", "address": "Apple st 652"},
9         {"name": "Hannah", "address": "Mountain 21"},
10        {"name": "Michael", "address": "Valley 345"}
11    ])
12
13 print(list(collection.find(*args: {}, { "_id": 0, "name": 1 })))
```

Не  
подгружаем  
поле \_id

Подгружаем поле  
"name"  
Остальные поля  
не подгружаются

```
3 live-coding\get_part_columns.py"
[{'name': 'Amy'}, {'name': 'Hannah'}, {'name': 'Michael'}]
```



# mongoDB



query\_example.py ×

```
1 from pymongo import MongoClient
2
3 if __name__ == '__main__':
4     client = MongoClient(host="mongodb://root:root@localhost", port=27017)
5     db = client["database"]
6     collection = db["collection_10"]
7     collection.insert_many([
8         {"model": "BMW", "cost": 100},
9         {"model": "Mercedes", "cost": 80},
10        {"model": "Ferrari", "cost": 200}
11    ])
12
13 print(list(collection.find(
14     "cost":{"$gt": 90, "$lt": 110}
15     })))
```

Фильтруем по  
полю cost  
>90 <110

3 live-coding\query\_example.py"

[{'\_id': ObjectId('67ce6234582ffdfaf13e6a77'), 'model': 'BMW', 'cost': 100}]

# MongoDB

\$eq: значение поля документа равно заданному

\$ne: значение поля документа не равно заданному

\$gt: значение поля документа больше заданного

\$gte: значение поля документа больше или равно заданному

\$lt: значение поля документа меньше заданного

\$lte: значение поля документа меньше или равно заданному

\$in: значение поля документа находится в выбранном наборе

\$and: возвращает документы, для которых выполнено оба условия

\$or: возвращает документы, для которых выполнено хотя бы одно условие

\$not: возвращает документы, для которых не выполняется условие

\$regex: выполнять сравнение значений полей объектов с использованием регулярных выражений

\$text: выполнить полнотекстовый поиск

\$where: использовать JavaScript выражение при сравнении полей объектов



# mongoDB



```
query_operator_in.py x
1 from pymongo import MongoClient
2
3 if __name__ == '__main__':
4     client = MongoClient(host="mongodb://root:root@localhost", port=27017)
5     db = client["database_random"]
6     collection = db["collection"]
7     collection.insert_many(
8         [{"item": 'Pens', "quantity": 350, "tag": 'school'},
9          {"item": 'Erasers', "quantity": 15, "tag": 'office'},
10         {"item": 'Books', "quantity": 5, "tag": 'home'}])
11
12     print(list(collection.find(
13         *args: {"tag": {"$in": ["school", "office"]}},
14         { "_id": 0, "item": 1, "tag": 1 })))
```

Выбираем те  
документы, у которых  
поле тег принимает  
одно из значений

```
[{'item': 'Pens', 'tag': 'school'}, {'item': 'Erasers', 'tag': 'office'}]
```

```
query_logical_operator.py x
15 print(list(collection.find(
16     *args: {"$or": [
17         {"$and": [{"quantity": {"$gt": 10}}, {"tag": "office"}]},
18         {"quantity": {"$lt": 10}}]),
19     {"_id": 0, "item": 1}
20 ))))
```

Оператор  
«ИЛИ»

Оператор  
«И»

```
[{'item': 'Erasers'}, {'item': 'Books'}]
```

quantity > 10 И tag == office

ИЛИ quantity < 10



mongoDB

```
random_query.py x
21
22 print(list(collection.find(
23     *args: {"district": 3, "$expr": { "$lt": [0.5, {"$rand": {} } ] }},
24     { "_id": 0, "name": 1}
25 )))
```

Будут выбраны только те документы, для которых функция rand вернёт значение меньше 0.5

```
[{'name': 'Beckham'}, {'name': 'Debarge'}, {'name': 'Grimwald'}, {'name': 'Justo'}, {'name': 'Archibald'},  
{ 'name': 'Debarge'}, {'name': 'Grimwald'}]
```



# mongoDB

```
query_count.py ×  
1 from pymongo import MongoClient  
2  
3 ▢ if __name__ == "__main__":  
4     client = MongoClient(host="mongodb://root:root@localhost", port=27017)  
5     db = client["database_count"]  
6     collection = db["collection"]  
7     collection.insert_many(  
8         [{"item": "Pens", "quantity": 350, "tag": "school", "cost": 100},  
9         {"item": "Erasers", "quantity": 15, "tag": "office", "cost": 50},  
10        {"item": "Books", "quantity": 5, "tag": "home", "cost": 400},  
11        {"item": "Rulers", "quantity": 20, "tag": "home", "cost": 30},  
12    ])  
13 print(collection.count_documents({"tag": "home"}))
```

Результат

Функция подсчёта  
количества документов

Запрос, по которому  
осуществляется отбор значений

```
MongoDB\Theme 3 live-coding\query_count.py"  
2
```



# mongoDB





# MongoDB. Обновление документов.

update\_field.py ×

```
1 from pymongo import MongoClient
2
3 if __name__ == '__main__':
4     client = MongoClient(host="mongodb://root:root@localhost", port=27017)
5     db = client["database"]
6     collection = db["collection_12"]
7     collection.insert_many([
8         {"model": "BMW", "cost": 100},
9         {"model": "Mercedes", "cost": 80},
10        {"model": "Ferrari", "cost": 200}
11    ])
12    collection.update_one(filter={"model": "BMW"}, update={"$set":{"cost": 90}})
13    print(list(collection.find({})))
```

update\_one – обновляет  
первое вхождение  
update\_many – обновляет все  
вхождения

Фильтр, по которому  
выбираем документы,  
поля которых обновляем

Оператор обновления  
поля

Задаем значение поля

Результат обновления  
поля

```
3 live-coding\update_field.py"
[{'_id': ObjectId('67ce67076951f9d973aa008b'), 'model': 'BMW', 'cost': 90}, {'_id': ObjectId('67ce67076951f9d973aa008c'), 'model': 'Mercedes', 'cost': 80}, {'_id': ObjectId('67ce67076951f9d973aa008d'), 'model': 'Ferrari', 'cost': 200}]
```



# MongoDB. Удаление документов.

delete\_document.py x

```
1 from pymongo import MongoClient
2
3 if __name__ == '__main__':
4     client = MongoClient(host="mongodb://root:root@localhost", port=27017)
5     db = client["database"]
6     collection = db["collection_15"]
7     collection.insert_many([
8         {"model": "BMW", "cost": 100},
9         {"model": "Mercedes", "cost": 80},
10        {"model": "Ferrari", "cost": 200}
11    ])
12    print(list(collection.find({})))
13    collection.delete_one({"model": "BMW"})
14    print(list(collection.find({})))
```

delete\_one – удаляет первое  
вхождение  
delete\_many удаляет все  
вхождения

Фильтруем по  
полю model

Документ BMW  
успешно удалён

```
[{'_id': ObjectId('67ce697312a3791a77f56453'), 'model': 'BMW', 'cost': 100}, {'_id': ObjectId('67ce697312a3791a77f56454'), 'model': 'Mercedes', 'cost': 80}, {'_id': ObjectId('67ce697312a3791a77f56455'), 'model': 'Ferrari', 'cost': 200}]
[{'_id': ObjectId('67ce697312a3791a77f56454'), 'model': 'Mercedes', 'cost': 80}, {'_id': ObjectId('67ce697312a3791a77f56455'), 'model': 'Ferrari', 'cost': 200}]
```

# goDB

При помощи MongoDB реализовать backend для приложения учёта просмотренных фильмов. Реализовать возможность добавления, удаления, обновления и получения фильмов из БД.

Каждый фильм хранить в виде документа со следующими полями:

- Название фильма
- Студия
- Год съёмки фильма
- Оценка
- Статус (просмотрено/нет)
- Список актёров
- Режиссёр
- Жанр

Реализовать возможность выборки и подсчёта числа фильмов по критериям:

- Год съёмки фильма (диапазон лет)
- Оценка (от n и выше)
- Выбранный актёр снимался в фильме
- Фильмы выбранного режиссёра
- Фильм указанного жанра
- Статус: просмотрено/нет
- Комбинации указанных критериев





пермский  
политех

# Базовые информационные процессы и технологии

## Тема №3 MongoDB

Докладчик:  
Нетбай Г.В., ведущий инженер кафедры ВММБ